

APPENDICES

CODE FOR FINGER PRINT AND KEYPAD SENSOR

```
#include <Adafruit_Fingerprint.h>
#include <SoftwareSerial.h> #include
<Wire.h> #include
<math.h>

#include <Bonezegei_LCD1602_I2C.h>

#include <Keypad.h>

int z1=0; int
y1=0;

const int ADXL345_ADDR = 0x53; int16_t x,
y, z;

long xSum = 0, ySum = 0, zSum = 0; int
sampleCount = 100;

float xyAngle = 0;

// LCD setup Bonezegei_LCD1602_I2C
lcd(0x27);

// SoftwareSerial for fingerprint sensor communication (pins 2 and 3) SoftwareSerial
mySerial(2, 3);

Adafruit_Fingerprint finger = Adafruit_Fingerprint(&mySerial); const
char* correctPasscode = "1234"; // Predefined 4-digit passcode String
enteredPasscode = ""; // Store user input

// Define the rows and columns of the keypad
```

```

const byte ROW_NUM      = 4; // Four rows

const byte COL_NUM      = 4; // Four columns

char keys[ROW_NUM][COL_NUM] = {

  { '1', '2', '3', 'A' },

  { '4', '5', '6', 'B' },

  { '7', '8', '9', 'C' },

  { '*', '0', '#', 'D' }

};

// Define the row and column pins on the Arduino byte

pin_rows[ROW_NUM] = {11, 10, 9, 8};

byte pin_cols[COL_NUM] = {7, 6, 5, 4};

// Create an instance of the Keypad library

Keypad keypad = Keypad(makeKeymap(keys), pin_rows, pin_cols, ROW_NUM,
COL_NUM);

void setup() {

  // Start serial communication

  Serial.begin(9600); Wire.begin();

  while (!Serial); // For Yun/Leo/Micro/Zero/... delay(100);

  // Serial.println("\n\nAdafruit Fingerprint sensor");

  Wire.beginTransmission(ADXL345_ADDR);

  Wire.write(0x2D);

  Wire.write(0x08); Wire.endTransmission();

  lcd.begin(); lcd.print("FINGER PRINT&

PW");

```

```

lcd.setPosition(0, 1);

lcd.print("BASED ARM"); //param1 = X          param2 = Y

delay(2000);

lcd.setPosition(0, 0);

//Serial.println("ADXL345 Initialized");

delay(500);

// Initialize the fingerprint sensor

finger.begin(57600);

if (finger.verifyPassword()) {

lcd.clear();  lcd.print("Sensor

Found!");

} else {

lcd.clear();

lcd.print("Sensor Error");

while (1); // Stop if the sensor is not found

}

delay(1000); void

loop() {

char key = keypad.getKey();

lcd.clear(); lcd.setPosition(0, 0);

lcd.print("Enter Password");

lcd.setPosition(0, 1);

lcd.print(enteredPasscode.c_str());

delay(100); if

(key) {

```

```

// Serial.print("Key Pressed: ");

//Serial.println(key);

// Handle special keys for input
if (key == '#') { // Submit the entered code checkPasscode();
}

else if (key == '*') { // Clear input (if needed)
enteredPasscode = "";
// Serial.println("Input cleared.");
}

else { // Add pressed key to the input
if (enteredPasscode.length() < 4) { // Only accept 4 digits
enteredPasscode += key;
// Serial.print("Entered: ");
//Serial.println(enteredPasscode);
}
}
}

while(z1)
{
verifyFingerprint();
}

while(y1)
{
lcd.clear();

```

```

lcd.setPosition(0, 0); lcd.print("PICK
AND"); lcd.setPosition(0, 1);
lcd.print("PLACE");          //param1 = X  param2 = Y
delay(100);
calculateAverage();
calculateXYAngle();
compareValues(); flex();
}

delay(100);
}

void verifyFingerprint() { int p
= -1;

//Serial.println("Place Finger to Verify");

// Wait for a valid finger image while (p
!= FINGERPRINT_OK) {
p = finger.getImage(); switch (p)
{
case FINGERPRINT_OK:
lcd.clear(); lcd.print("Image
taken"); break;
case FINGERPRINT_NOFINGER:
lcd.clear(); lcd.print("No
Finger"); break;

```

```

case FINGERPRINT_PACKETRECEIVEERR:

lcd.clear(); lcd.print("Comm

Error"); break;

case FINGERPRINT_IMAGEFAIL:

lcd.clear(); lcd.print("Image

Error"); break;

default:

lcd.clear(); lcd.print("Unknown

Error"); break;

}

delay(100);

}

// Convert image to template and search for a match p =

finger.image2Tz();

if (p == FINGERPRINT_OK) {

p = finger.fingerSearch();

if (p == FINGERPRINT_OK) {

lcd.clear(); lcd.print("Access

Granted"); lcd.setPosition(0, 1);

lcd.print("USE UR GLOVE");

//lcd.print(finger.fingerID); y1=1;

z1=0;

}

else      {

y1=0;

```

```

z1=0;

lcd.clear();

lcd.print("No Match Found");

}

} else {

lcd.clear();

lcd.print("Error Converting");

}

delay(2000); // Wait for 2 seconds before verifying again

}

void checkPasscode() {

// Compare the entered passcode with the correct passcode if

(enteredPasscode == correctPasscode) {

z1=1;

lcd.clear();

lcd.setPosition(0, 0);

lcd.print("PASSWORD OK");

delay(2000);

lcd.clear();

lcd.setPosition(0, 0);

lcd.print("PLACE FINGER ");

lcd.setPosition(0, 1);

lcd.print("ON SENSOR");           //param1 = X  param2 = Y

delay(2000);

```

```

//Serial.println("\nPasscode Correct! Access Granted.")

} else {
z=0;
//Serial.println("\nIncorrect Passcode. Access Denied.");
}

// Reset entered passcode after check enteredPasscode = "";
//Serial.println("Please enter the 4-digit passcode again.");
}

void calculateAverage() { xSum = 0; ySum
= 0;
zSum = 0;

for (int i = 0; i < sampleCount; i++) { readADXL345(); xSum
+= x; ySum += y; zSum += z; delay(10);
}

x = xSum / sampleCount;

y = ySum / sampleCount;

z = zSum / sampleCount;

}

void readADXL345(

{ Wire.beginTransaction(ADXL345_ADDR); Wire.write(0x32);
Wire.endTransmission(false); Wire.requestFrom(ADXL345_ADDR, 6, true); x =
(Wire.read() | (Wire.read() << 8));
y = (Wire.read() | (Wire.read() << 8));
z = (Wire.read() | (Wire.read() << 8));

}

```



```

void calculateXYAngle() {

xyAngle = atan2(y, x) * 180.0 / M_PI; if (xyAngle < 0) {
xyAngle += 360;
}

}

void compareValues() {

if (abs(x) > abs(y) && abs(x) > abs(z)) { if (x > 0) {

//Serial.println("Orientation: Right Tilt"); lcd.setPosition(0, 1); lcd.print("Right Tilt ");
Serial.println("D");

} else {

// Serial.println(

Orientation: Left Tilt"); lcd.setPosition(0, 1); lcd.print("Left Tilt "); Serial.println("C");

}

} else if (abs(y) > abs(x) && abs(y) > abs(z)) { if (y > 0) {

// Serial.println("Orientation: Front Tilt"); lcd.setPosition(0, 1); lcd.print("Front Tilt ");
Serial.println("A");

} else {

// Serial.println("Orientation: Back Tilt"); lcd.setPosition(0, 1); lcd.print("Back Tilt ")
Serial.println("B");

}

} else if (abs(z) > abs(x) && abs(z) > abs(y)) { if (z > 0) {

// Serial.println("Orientation: Normal"); lcd.setPosition(0, 1); lcd.print("Normal ");

} else {

```

```

// Serial.println("Orientation: Upside Down"); lcd.setPosition(0, 1); lcd.print("Upside
Down");

}

} else {

//Serial.println("Orientation: Unknown"); lcd.setPosition(0, 1); lcd.print("Unknown ");

}

}

void flex()

{

float v = analogRead(A0);

// Convert the analog reading (which goes from 0 - 1023) to a voltage (0 - 5V): float v1
= v * (5.0 / 1023.0);

// print out the value you read:

Serial.println(v1);

// Wait for a short period before taking the next reading delay(100);

if(v1>1.2)

{

Serial.println("F");

}

else

{

Serial.println("E")

```

CODE FOR SERVO MOTOR AND GRIPPER

```
#include <Servo.h>

// Declare servo objects for each axis

Servo servo1; // Left-Right rotation (Servo 1)

Servo servo2; // Front-Back arm movement (Servo 2) Servo servo3; // Gripper open-
close (Servo 3)

// Initialize positions for each servo

int pos1 = 90; // Default position for Servo 1 (Left-Right) int pos2 = 90; // Default
position for Servo 2 (Front-Back) int pos3 = 90;
// Default position for Servo 3 (Gripper) void setup() {

Serial.begin(9600); // Start serial communication at 115200 baud rate

// Attach the servos to the corresponding pins on NodeMCU

servo1.attach(D0); // Left-Right rotation servo on pin D2 servo2.attach(D1); // Front-
Back movement servo on pin D3 servo3.attach(D2); // Gripper servo on pin D4

// Initialize servos to default positions servo1.write(pos1); servo2.write(pos2);
servo3.write(pos3);

}

void loop() {

// Check if there is any incoming data from Serial if (Serial.available() > 0) { char
command = Serial.read(); // Read the incoming byte as a command

// Handle the serial commands for different actions if (command == 'A') { // Front
movement

pos2 = min(pos2 + 10, 180); // Increase position to move the arm front (max 180
degrees)

servo2.write(pos2);
```

```

}

else if (command == 'B') { // Back movement

pos2 = max(pos2 - 10, 0);      // Decrease position to move the arm back (min 0
degrees)

servo2.write(pos2);

}

else if (command == 'C') { // Left rotation

pos1 = max(pos1 - 10, 0);      // Rotate left (min 0 degrees) servo1.write(pos1);

}

else if (command == 'D') { // Right rotation

pos1 = min(pos1 + 10, 180); // Rotate right (max 180 degrees) servo1.write(pos1);

}

else if (command == 'E') { // Open gripper

pos3 = 180; // Set servo to open position (fully open) servo3.write(pos3); else

if (command == 'F') { // Close gripper

pos3 = 68; // Set servo to close position (fully closed) servo3.write(pos3);

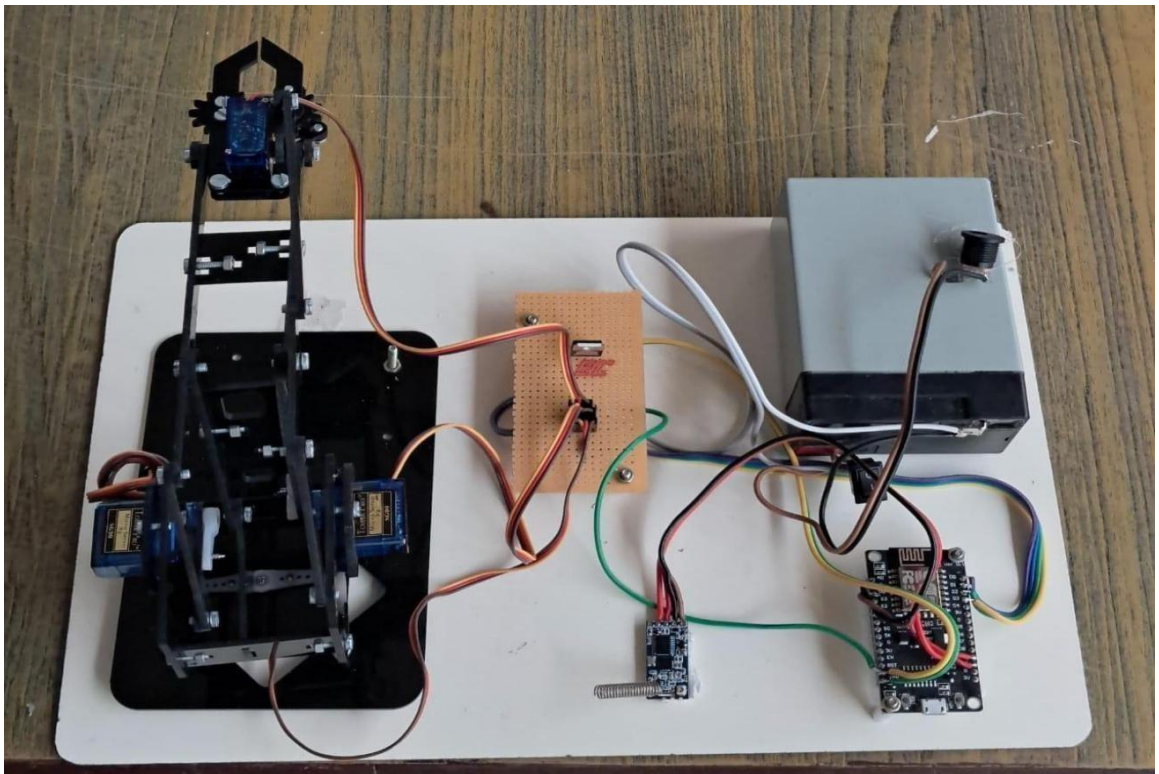
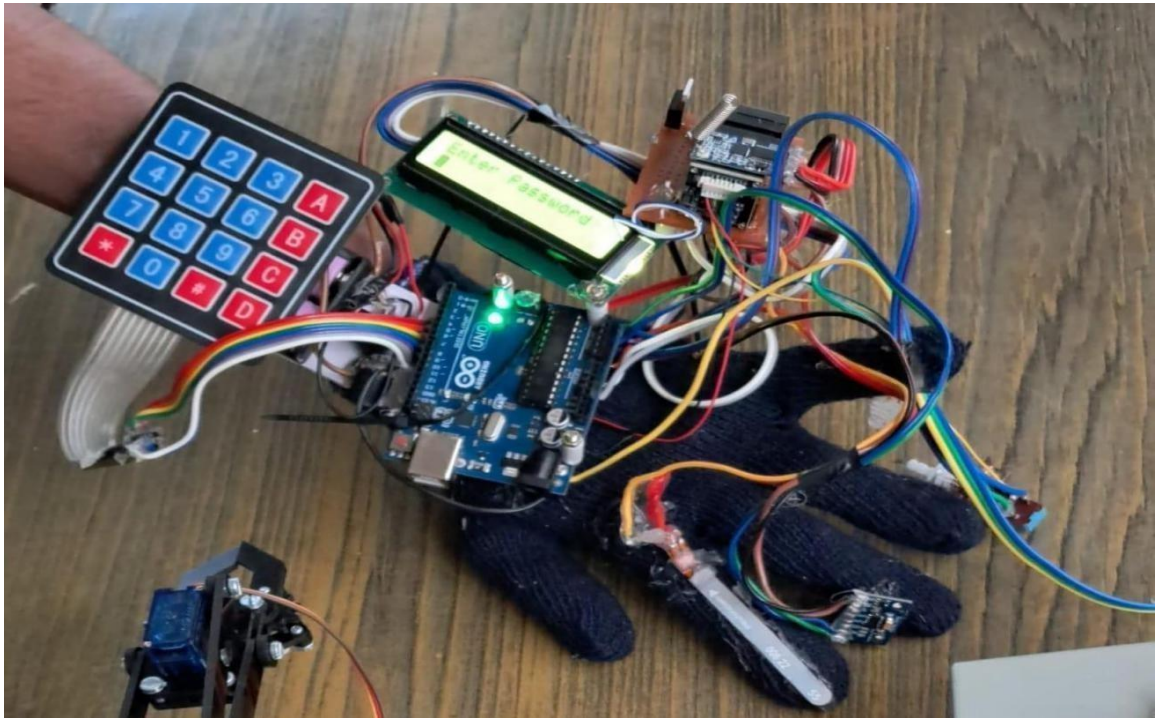
}

delay(50); // Small delay to allow for servo movement

}

```

PHOTOGRAPHS



PHOTOGRAPHS

